



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Materia: Estructura de Datos y Algoritmos II

Proyecto: Visualizador Interactivo de Algoritmos

Reporte Final de Proyecto

Documentación, Arquitectura e Integración

Documentación elaborada por:

Equipo de Desarrollo — Backend
(Francisco José Coutiño Morales y equipo)

Periodo: 2025 – 2026

Índice

1. Descripción General del Proyecto	2
2. Arquitectura del Sistema	2
3. Módulos Implementados	3
3.1. Backend — Algoritmos	3
3.2. Chatbot RAG	3
3.3. Frontend	3
4. Proceso de Desarrollo e Integración Backend–Chatbot	3
4.1. Migración de LLM: Google Gemini → Groq	4
4.2. Migración de Loader: GenericLoader → DirectoryLoader	4
4.3. Eliminación de ChromaDB como vector store	4
4.4. Mejoras al sistema de búsqueda	4
4.5. Actualización de dependencias	4
5. Advertencias Críticas y Prevención de Fallos	5
6. URLs de Producción	6
7. Créditos del Proyecto	7
7.1. Equipo de Backend	7
7.2. Equipo de Frontend	7
7.3. Equipo de Chatbot IA	7
8. Tecnologías Utilizadas	8
9. Observaciones Finales	8

1. Descripción General del Proyecto

El **Visualizador Interactivo de Algoritmos** es un sistema web interactivo diseñado como proyecto final de la materia *Estructura de Datos y Algoritmos II*. Su propósito es doble: servir como herramienta de visualización pedagógica para estudiantes de ingeniería y como demostración de arquitectura de software moderna aplicada al ámbito académico.

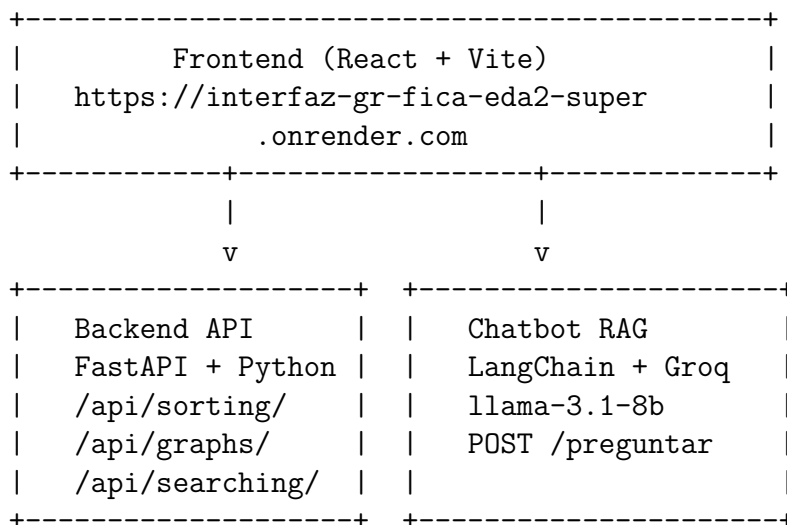
El sistema integra tres componentes desplegados de forma independiente en producción:

- **Backend API REST** desarrollado en Python con FastAPI, que implementa y expone todos los algoritmos del temario de la materia.
- **Interfaz Gráfica Web** desarrollada en React + Vite con estética Frutiger Aero, que permite la interacción visual con los algoritmos.
- **Chatbot RAG (Asistente IA)** basado en LangChain + Groq, que responde preguntas sobre los algoritmos implementados en el proyecto utilizando el código fuente del backend como base de conocimiento.

2. Arquitectura del Sistema

Repositorio Frontend: [Interfaz-gr-fica-EDA2-SUPER](#)

Repositorio Backend: [Proyecto-EDAI-Backend](#)



Los tres servicios se despliegan de forma gratuita en **Render.com**. El Frontend se sirve como **Static Site**, el Backend y el Chatbot como **Web Services**.

Módulo	Algoritmos
Ordenamiento	Bubble Sort, Heap Sort, Quick Sort, Counting Sort, Radix Sort, Merge Sort
Búsqueda	Lineal, Binaria, Hash con encadenamiento
Grafos	Representación (Matriz/Lista), BFS, DFS
Árboles	Árbol Binario, Inorden/Preorden/Postorden, Notaciones
Paralelos	Reducción paralela, Multiplicación de matrices

3. Módulos Implementados

3.1. Backend — Algoritmos

3.2. Chatbot RAG

El chatbot utiliza la técnica de **Retrieval-Augmented Generation (RAG)**:

1. Al recibir una pregunta, clona la rama `dev` del repositorio backend.
2. Carga los archivos `.py` de la carpeta `app/algoritmos/` con `DirectoryLoader`.
3. Realiza búsqueda por palabras clave con normalización (`camelCase`, traducciones español $\rightarrow$ inglés, variaciones con guion bajo).
4. Pasa los fragmentos relevantes como contexto al LLM (`Groq llama-3.1-8b-instant`).
5. Retorna la respuesta al frontend vía `POST /preguntar`.

3.3. Frontend

Interfaz de una sola página (SPA) con:

- Navegación por secciones: Inicio, Algoritmos, Documentación, Nosotros, Asistente.
- Índice Temático Interactivo con `Scrollspy` de Bootstrap.
- Visor de PDF embebido con navegación por temas.
- Ventana de chat flotante conectada al chatbot en tiempo real.
- Estética Frutiger Aero con animaciones y música de fondo.

4. Proceso de Desarrollo e Integración Backend–Chatbot

El desarrollo del chatbot implicó una colaboración directa entre los equipos de Backend y Chatbot para lograr la integración funcional del sistema RAG. A continuación se documenta el proceso de cambios técnicos realizados:

4.1. Migración de LLM: Google Gemini → Groq

El LLM original (ChatGoogleGenerativeAI con `gemini-1.5-flash`) fue reemplazado por ChatGroq con `llama-3.1-8b-instant` debido a restricciones de red en Render (plan gratuito) que bloqueaban las conexiones salientes hacia las APIs de Google. El modelo `llama3-8b-8192` fue discontinuado por Groq durante el desarrollo, migrándose a `llama-3.1-8b-instant`.

Archivo modificado: `chatbot_logic/chain.py`

4.2. Migración de Loader: GenericLoader → DirectoryLoader

`GenericLoader.from_path` no está disponible en la versión de `langchain-community` desplegada. Se reemplazó por `DirectoryLoader` con `PythonLoader`, que carga directamente todos los archivos `.py` de forma recursiva sin dependencias de parsers adicionales.

Archivo modificado: `chatbot_logic/loader.py`

4.3. Eliminación de ChromaDB como vector store

La dependencia de ChromaDB requería credenciales para los embeddings que no estaban disponibles en el entorno de Render. Se reemplazó el sistema de recuperación vectorial por una **búsqueda directa por palabras clave** sobre los documentos cargados en memoria, con caché de sesión para evitar clonar el repositorio en cada petición.

Archivo modificado: `chatbot_logic/retriever.py`

4.4. Mejoras al sistema de búsqueda

- **Normalización de camelCase:** `MergeSort` → `merge_sort` mediante expresiones regulares.
- **Diccionario de traducciones español → inglés:** permite consultas en español que encuentren código en inglés.
- **Búsqueda en metadata de archivos:** además del contenido, se busca en el nombre del archivo fuente (`source`).
- **Variaciones de separadores:** genera variantes con espacio, guion bajo y sin separador.

4.5. Actualización de dependencias

<code>langchain-google-genai</code>	->	<code>langchain-groq==0.2.0</code>
<code>llama3-8b-8192</code>	->	<code>llama-3.1-8b-instant</code>
<code>GenericLoader</code>	->	<code>DirectoryLoader + PythonLoader</code>
<code>ChromaDB (vectores)</code>	->	<code>Busqueda por palabras clave en memoria</code>

5. Advertencias Críticas y Prevención de Fallos

5.1 Advertencias Lógicas (Degradación de servicio)

Estas condiciones no interrumpen el sistema pero pueden afectar la calidad de las respuestas o el rendimiento de la plataforma:

- **Respuesta incompleta del Chatbot:** El bot responde "no tengo información" sobre un algoritmo existente. *Causa:* La consulta usa términos no cubiertos por el diccionario de traducciones o el retriever no encontró coincidencias.
- **Latencia alta en el primer arranque:** La primera respuesta del chatbot tarda 60–90 segundos. *Causa:* El servicio de Render (plan gratuito) se duerme tras 15 minutos de inactividad; el tiempo de arranque frío incluye clonar el repositorio de GitHub completo.
- **Alucinación parcial de código:** El chatbot genera código que no coincide exactamente con el del proyecto. *Causa:* El LLM complementa con conocimiento general cuando el contexto RAG recuperado es insuficiente.
- **Navegación incorrecta en visor PDF:** El visor no muestra la página correcta al cambiar de tema. *Causa:* Algunos navegadores no respetan el parámetro `#page=N` en iframes por políticas de seguridad.
- **Timeout ocasional (HTTP 503):** El backend responde con un error 503. *Causa:* Render reinicia instancias gratuitas periódicamente; el primer request tras el reinicio activa el arranque frío.

5.2 Errores Fatales (Fin abrupto del sistema)

Estas condiciones causan la caída total del servicio afectado y requieren intervención técnica inmediata:

- **GROQ_API_KEY no configurada o expirada:** *Afecta a: Chatbot.* La variable de entorno fue eliminada o la key fue revocada. **Solución:** Generar nueva key en `console.groq.com` y actualizar en el dashboard de Render.
- **Repositorio backend o rama dev eliminada:** *Afecta a: Chatbot.* El loader clona `dev` al iniciar; si no existe, falla la inicialización del RAG. **Solución:** Restaurar la rama `dev` en el repositorio original.
- **ImportError en chatbot_logic/:** *Afecta a: Chatbot.* Actualización incompatible de una dependencia de LangChain durante el redesplicue. **Solución:** Fijar versiones exactas en el `requirements.txt`.
- **Build de Vite falla por error de sintaxis JSX:** *Afecta a: Frontend.* Caracteres especiales con codificación incorrecta al editar `App.jsx`. **Solución:** Verificar que el archivo se guarde en UTF-8 puro.
- **Puerto no detectado por Render:** *Afecta a: Backend/Chatbot.* El Procfile no expone correctamente `$PORT`. **Solución:** Verificar `uvicorn app.main:app -host 0.0.0.0 -port $PORT` en el Procfile.
- **Conflicto de merge no resuelto en App.jsx:** *Afecta a: Frontend.* Push simultáneo de múltiples desarrolladores. **Solución:** Establecer flujo de trabajo con *feature branches* y *pull requests*.

6. URLs de Producción

Servicio	URL
Frontend	https://interfaz-gr-fica-eda2-super.onrender.com
Backend API (docs)	https://proyecto-edaii-backend.onrender.com/docs
Chatbot	https://repo-chatbot-ia.onrender.com
Documentación Vol. I (PDF)	/docs/EDA2_Vol1_Codigo.pdf
Documentación Vol. II (PDF)	/docs/EDA2_Vol2_Marco_Teorico_1.pdf

7. Créditos del Proyecto

7.1. Equipo de Backend

Lead: Francisco José Coutiño Morales

- Francisco José Coutiño Morales (Lead Backend Developer)
- Ernesto Flamenco Villaseñor (Backend Developer)
- Tlalli Ortega González (Backend Developer)
- Sergio Alonso Salcedo Sainz (Backend Developer)
- Orlando Ocampo Lagunas (Backend Developer)
- Aarón Salazar Castillo (Backend Developer)
- Erick Cruz Solís (Backend Developer)
- Augusto Gandarilla Ibarra (Backend Developer)
- Neri Israel Cruz Cárdenas (Backend Developer)

7.2. Equipo de Frontend

Lead: González Baca María Fernanda

- González Baca María Fernanda
- Soriano Nieves Andrés
- Casanova Cortina América Daniela
- Toledo Reyes Vania
- Carlos Didier Cecilio Alejandro

7.3. Equipo de Chatbot IA

Lead: Jim Nava Bryan

- Jim Nava Bryan
- Martínez Mondragón Benjamín
- Arroyo Castañón Dulce Carolina
- Perea Yáñez Oscar
- Hernández Camargo Josué
- Galindo Contreras Ángel Gabriel
- Marín Pérez Daniela
- Escobedo Reyes Emma Esmeralda
- Martínez García Nour
- Jiménez Vega José Enrique
- Orozco González Samuel Alejandro
- Zambrano Mota María Fernanda

8. Tecnologías Utilizadas

Categoría	Tecnología
Backend	Python 3.12, FastAPI, Uvicorn, SQLite
Frontend	React 19, Vite 8, Bootstrap 5
Chatbot	LangChain 0.3, Groq (llama-3.1-8b-instant), GitPython
Despliegue	Render.com (Static Site + Web Services)
Control de versiones	Git, GitHub
Documentación	ReportLab, LaTeX

9. Observaciones Finales

El proyecto **Visualizador Interactivo de Algoritmos** cumple con todos los requisitos establecidos en el temario de *Estructura de Datos y Algoritmos II*: colecciones, paquetes, modificadores de acceso, herencia, polimorfismo, clases abstractas, interfaces, excepciones, E/S de archivos, GUI y concurrencia (crédito extra). La integración de un sistema RAG como chatbot añade un componente de inteligencia artificial aplicada que enriquece el valor didáctico y técnico del proyecto.

El proceso de desarrollo estuvo marcado por la colaboración interdisciplinaria entre tres equipos, la resolución iterativa de problemas de compatibilidad en entornos de producción y la toma de decisiones técnicas orientadas a la estabilidad del sistema dentro de las restricciones del plan gratuito de Render.

Documentación elaborada por el Equipo de Backend — Visualizador Interactivo de Algoritmos
Estructura de Datos y Algoritmos II | Facultad de Ingeniería, UNAM | 2026-2